



笔记分享平台

项目概述

本项目是一个基于**事件驱动架构（EDA）**的笔记分享平台，核心特点是通过自定义消息中间件实现完全解耦和异步事件处理。

核心设计理念：

- **Flask 不直接访问数据库**：所有数据库操作都通过中间件进行
- **生产者-消费者模式**：实现组件间的异步通信
- **发布-订阅模式**：支持多消费者订阅同一主题

一、软件架构设计

1.1 架构风格：事件驱动架构（EDA）

事件驱动架构是一种基于事件产生、检测、消费和响应的架构模式。本项目中：

角色	职责	实现组件
事件生产者	发布事件到中间件	Flask + MessageProducer
事件代理	路由和分发事件	MiddlewareServer
事件消费者	处理事件并执行业务逻辑	AuditConsumer, NoticeConsumer, StatConsumer, QueryConsumer

1.2 整体架构图

整体架构（最终版）

前端浏览器
(HTML/CSS)
- 表单提交
- 页面渲染

HTTP POST/GET

Flask Web 应用

路由层

GET /	→ index()
GET /note/<note_id>	→ note_detail()
GET /add	→ add_note() [展示表单]
POST /add	→ add_note() [处理提交]
POST /note/<id>/like	→ like_note()
POST /note/<id>/comment	→ comment_note()
POST /note/<id>/delete	→ delete_note_route()

QueryClient
(Python)

query/request

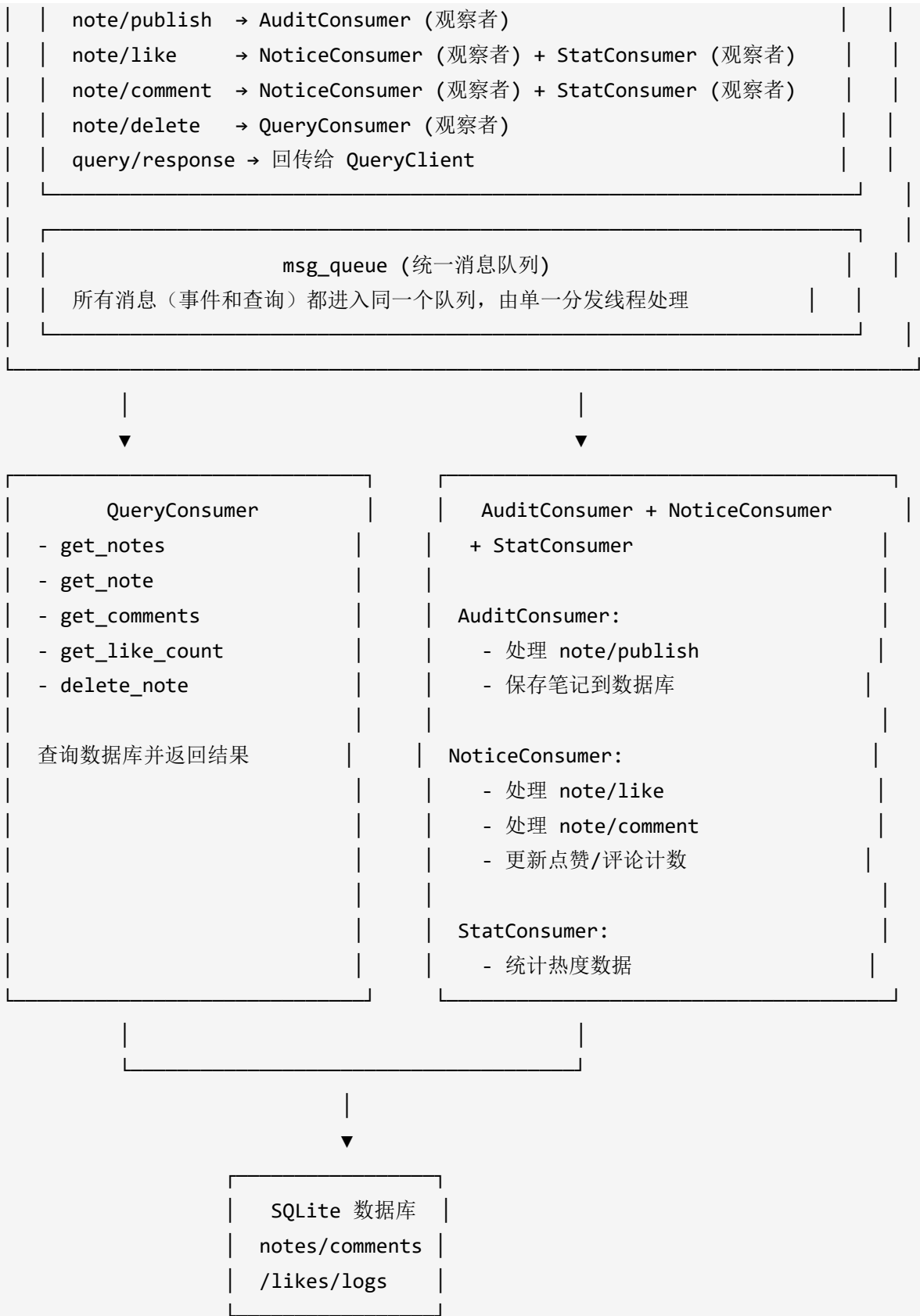
MessageProducer
(Python)

note/*

消息中间件 (MiddlewareServer)

TopicRouter (主题路由 - 观察者模式实现)

query/request → QueryConsumer (观察者)



1.3 组件职责详解

组件	层级	职责	是否访问数据库
Flask Web 服务器	表示层	处理 HTTP 请求、渲染 HTML 模板	✗ 否
MessageProducer	业务层	向中间件发布事件消息	✗ 否
QueryClient	业务层	通过中间件发送查询请求并接收响应	✗ 否
MiddlewareServer	中间件层	消息路由、队列管理、连接管理	✗ 否
AuditConsumer	数据层	审核并保存新发布的笔记	✓ 是
NoticeConsumer	数据层	处理点赞、评论事件	✓ 是
StatConsumer	数据层	更新笔记热度分数	✓ 是
QueryConsumer	数据层	处理数据库查询请求和删除事件	✓ 是
SQLite 数据库	存储层	持久化所有数据	-

二、中间件设计

2.1 核心组件

2.1.1 中间件服务器 (MiddlewareServer)

```
# 核心职责：
1. TCP Socket 监听（端口 5001）
2. 接收客户端连接（生产者、消费者、查询客户端）
3. 解析消息类型并路由
4. 管理统一消息队列
5. 维护主题订阅关系
6. 实现观察者模式：主题为被观察者，消费者为观察者
```

消息处理逻辑：

消息类型	处理方式	目标队列
query	查询请求	msg_queue
subscribe	订阅注册	TopicRouter
publish	事件发布	msg_queue

2.1.2 消息类 (Message)

```
# 数据结构
{
    "topic": "note/publish",      # 消息主题
    "content": {...},            # 消息内容
    "timestamp": "2024-01-01T12:00:00", # 创建时间
    "message_id": "uuid-xxx"      # 唯一标识
}

# 核心方法
- to_json(): 序列化为 JSON 字符串
- from_json(): 从 JSON 反序列化
```

2.1.3 消息队列 (MessageQueue)

```
# 设计特点
- 线程安全: 使用 threading.Lock 保证并发安全
- FIFO 顺序: 先进先出处理消息
- 阻塞获取: 队列为空时阻塞等待

# 核心方法
- put_message(message): 入队 (线程安全)
- get_message(): 出队 (线程安全, 阻塞)
- is_empty(): 检查队列是否为空
```

2.1.4 主题路由 (TopicRouter)

```
# 核心数据结构 - 实现观察者模式的订阅管理
subscribers = {
    "note/publish": [socket1, socket2], # AuditConsumer订阅
    "note/like": [socket3, socket4],    # NoticeConsumer + StatConsumer订阅
    "note/comment": [socket3, socket4],  # NoticeConsumer + StatConsumer订阅
    "query/request": [socket5]           # QueryConsumer订阅
}

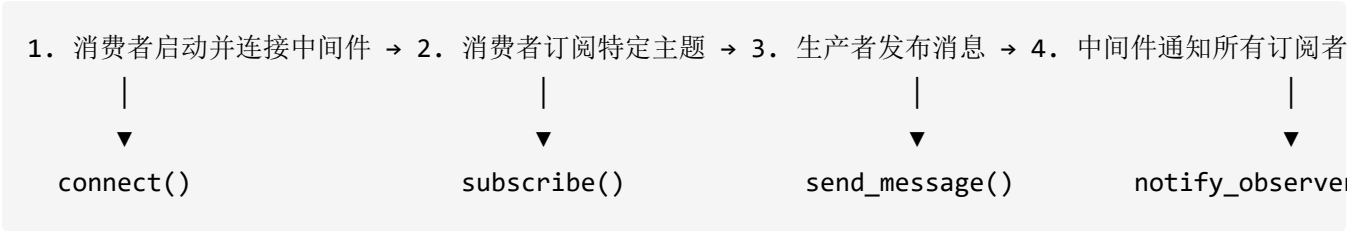
# 核心方法
- subscribe(topic, socket): 添加观察者（消费者订阅主题）
- unsubscribe(topic, socket): 移除观察者（消费者取消订阅）
- get_subscribers(topic): 获取订阅该主题的所有观察者（消费者）
- notify_observers(topic, message): 通知所有观察者（分发消息）
```

2.2 观察者模式的实现

2.2.1 角色映射

观察者模式概念	消息中间件实现
被观察者 (Subject)	主题 (Topic)
观察者 (Observer)	消费者 (Consumer)
添加观察者	消费者订阅主题 (subscribe_topic)
通知观察者	中间件向消费者推送消息

2.2.2 观察者模式流程



2.2.3 消息分发机制



2.2.4 单队列设计

队列	用途	分发线程
msg_queue	处理所有消息（事件消息和查询请求）	dispatch_messages

设计原因：

- 统一消息处理，简化架构
- 降低系统复杂度
- 避免队列间协调的复杂性

2.3 支持的消息主题

主题	生产者	消费者（观察者）	消息结构
note/publish	Flask	AuditConsumer	{note_id, title, content, author, crea
note/like	Flask	NoticeConsumer, StatConsumer	{note_id, user_id}
note/comment	Flask	NoticeConsumer, StatConsumer	{note_id, user_id, comment_text}
note/delete	Flask	QueryConsumer	{note_id}
query/request	QueryClient	QueryConsumer	{query_id, query_type, params}
query/response	QueryConsumer	QueryClient	{query_id, status, data}

2.4 消费者作为观察者的职责

2.4.1 AuditConsumer（审核消费者）

- **观察主题:** `note/publish`
- **观察者行为:** 接收笔记发布事件，验证并保存到数据库
- **业务逻辑:** 审核并保存新发布的笔记

2.4.2 NoticeConsumer（通知消费者）

- **观察主题:** `note/like` , `note/comment` , `note/delete`
- **观察者行为:** 接收用户互动事件，处理通知和计数
- **业务逻辑:** 处理点赞、评论和删除事件

2.4.3 StatConsumer（统计消费者）

- **观察主题:** `note/like` , `note/comment`
- **观察者行为:** 接收互动事件，更新统计信息
- **业务逻辑:** 更新笔记热度分数

2.4.4 QueryConsumer（查询消费者）

- **观察主题:** `query/request` , `note/delete`
 - **观察者行为:** 接收查询请求，执行数据库操作
 - **业务逻辑:** 执行数据库查询和删除操作
-

三、消费者设计

3.1 消费者基类 (MessageConsumer)

```
# 通用功能
```

- `__init__(host, port, component_name)`: 初始化消费者, 包含日志记录器
- `connect()`: 建立与中间件的 TCP 连接
- `subscribe_topic(topic)`: 订阅指定主题
- `start_listening()`: 启动消息监听线程
- `receive_message()`: 接收并解析消息 (处理 TCP 粘包)
- `process_message(message)`: 处理消息 (子类实现)
- `close()`: 关闭连接并清理资源
- `logger`: 日志记录器, 用于记录连接、订阅、消息处理等事件

3.2 各消费者职责

AuditConsumer (审核消费者)

- **订阅主题:** `note/publish`
- **职责:**
 - i. 接收笔记发布消息
 - ii. 验证笔记内容
 - iii. 保存到 `notes` 表
 - iv. 记录发布日志到 `publish_logs` 表
- **日志记录:** 记录连接状态、消息接收、处理结果等信息

NoticeConsumer (通知消费者)

- **订阅主题:** `note/like`, `note/comment`, `note/delete`
- **职责:**
 - i. **点赞:** 调用 `add_like()` + 更新笔记点赞数
 - ii. **评论:** 调用 `add_comment()` + 更新笔记评论数
 - iii. **删除:** 调用 `delete_note()` 删除笔记记录
- **日志记录:** 记录互动事件处理过程、通知生成等信息

StatConsumer (统计消费者)

- **订阅主题:** `note/like` , `note/comment`
- **职责:**
 - i. **点赞:** 热度 +2
 - ii. **评论:** 热度 +3
- **日志记录:** 记录热度更新过程、统计数据变化等信息

QueryConsumer (查询消费者)

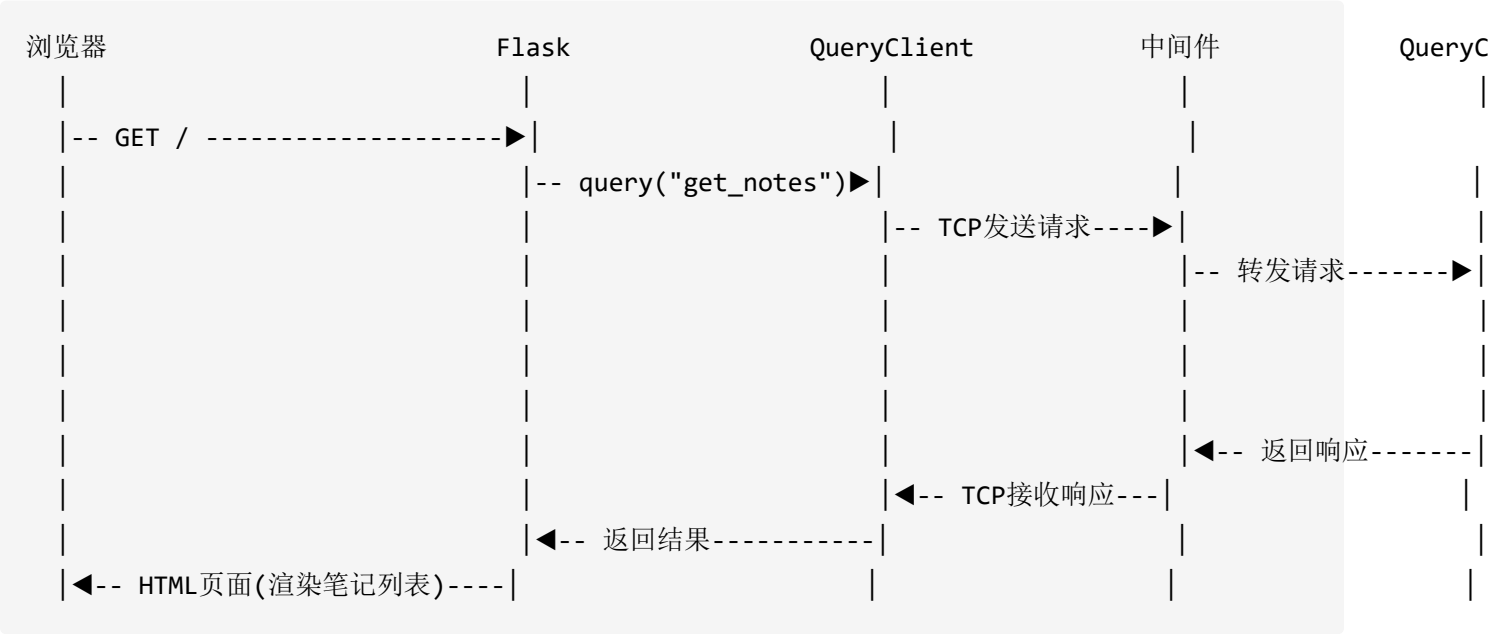
- **订阅主题:** `query/request` , `note/delete`
- **支持查询类型:**

查询类型	功能	参数
<code>get_notes</code>	获取所有笔记	无
<code>get_note</code>	获取单条笔记	<code>{note_id}</code>
<code>get_comments</code>	获取评论列表	<code>{note_id}</code>
<code>get_like_count</code>	获取点赞数	<code>{note_id}</code>

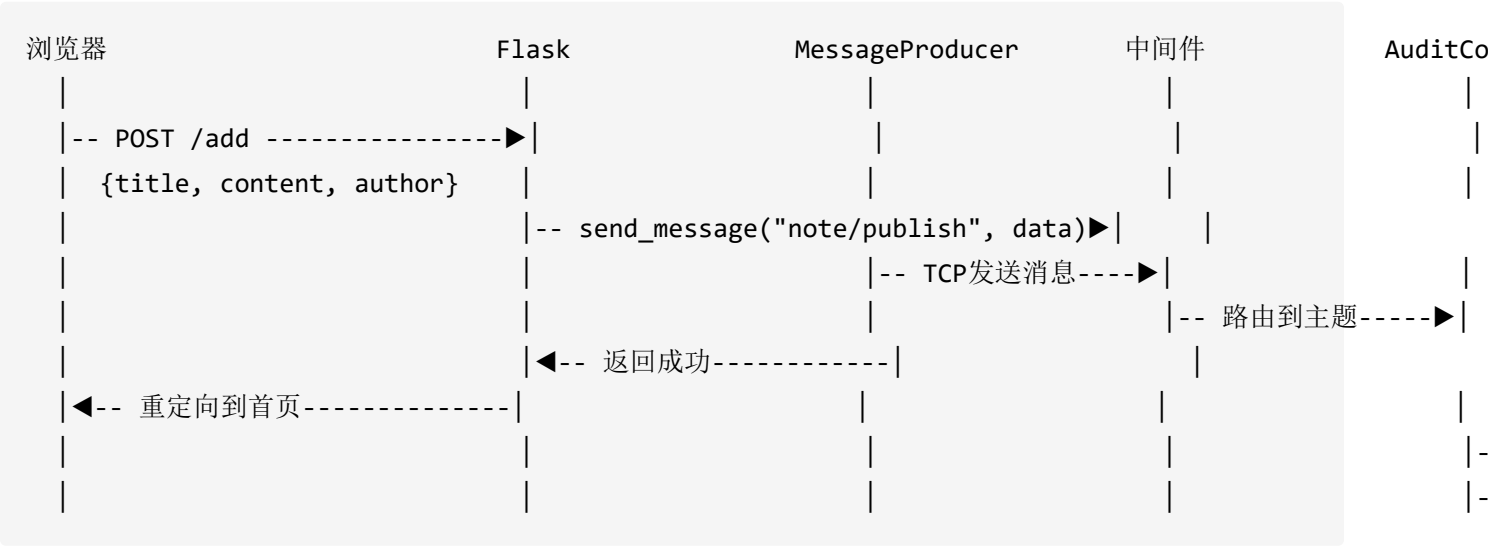
- **日志记录:** 记录查询请求处理、数据库操作、响应发送等信息
-

四、事件时序图

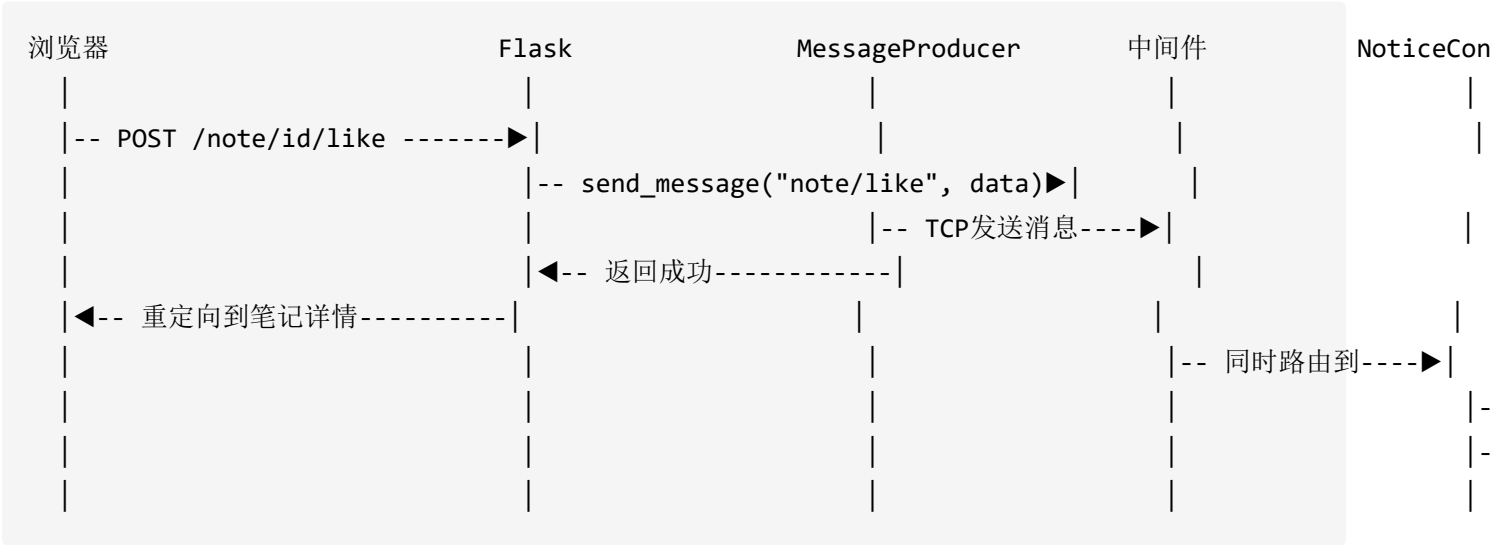
4.1 获取笔记列表（首页加载）



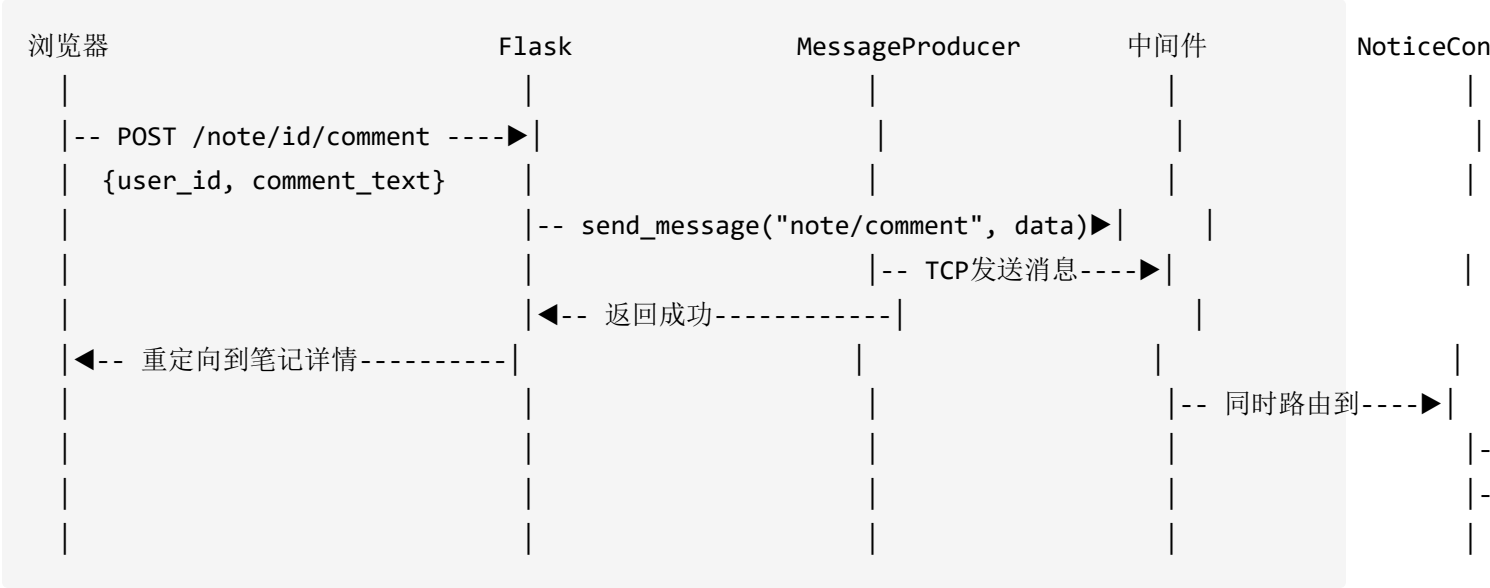
4.2 添加笔记（表单提交）



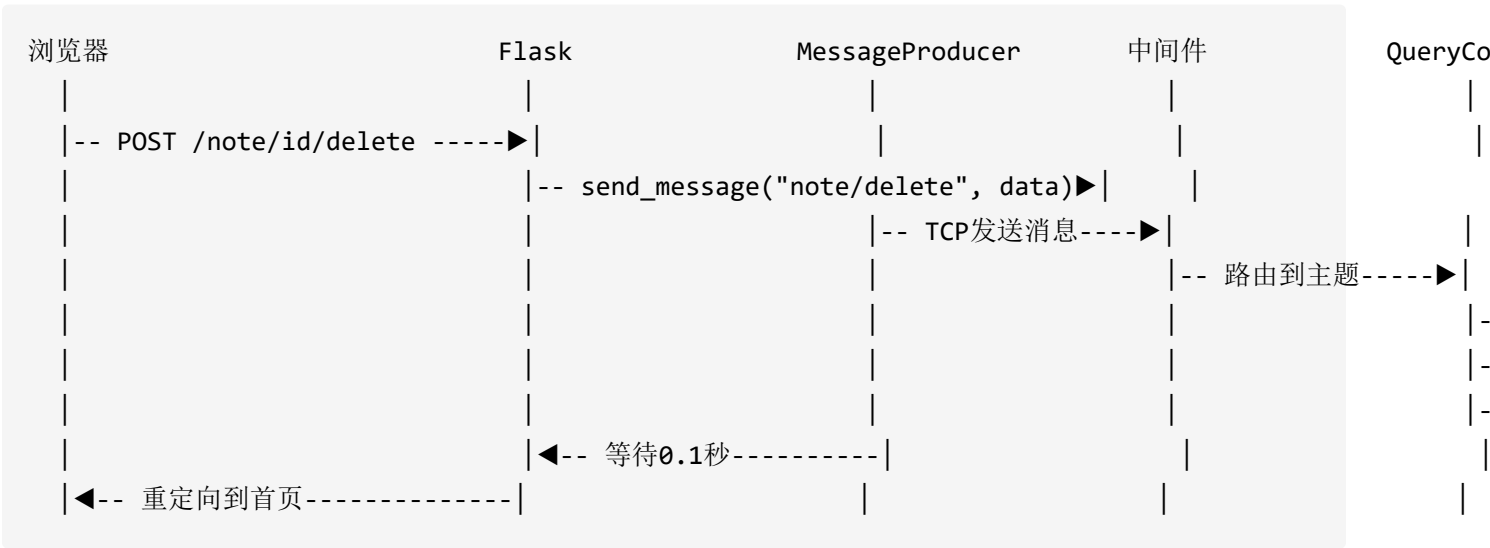
4.3 点赞笔记（表单提交）



4.4 评论笔记（表单提交）

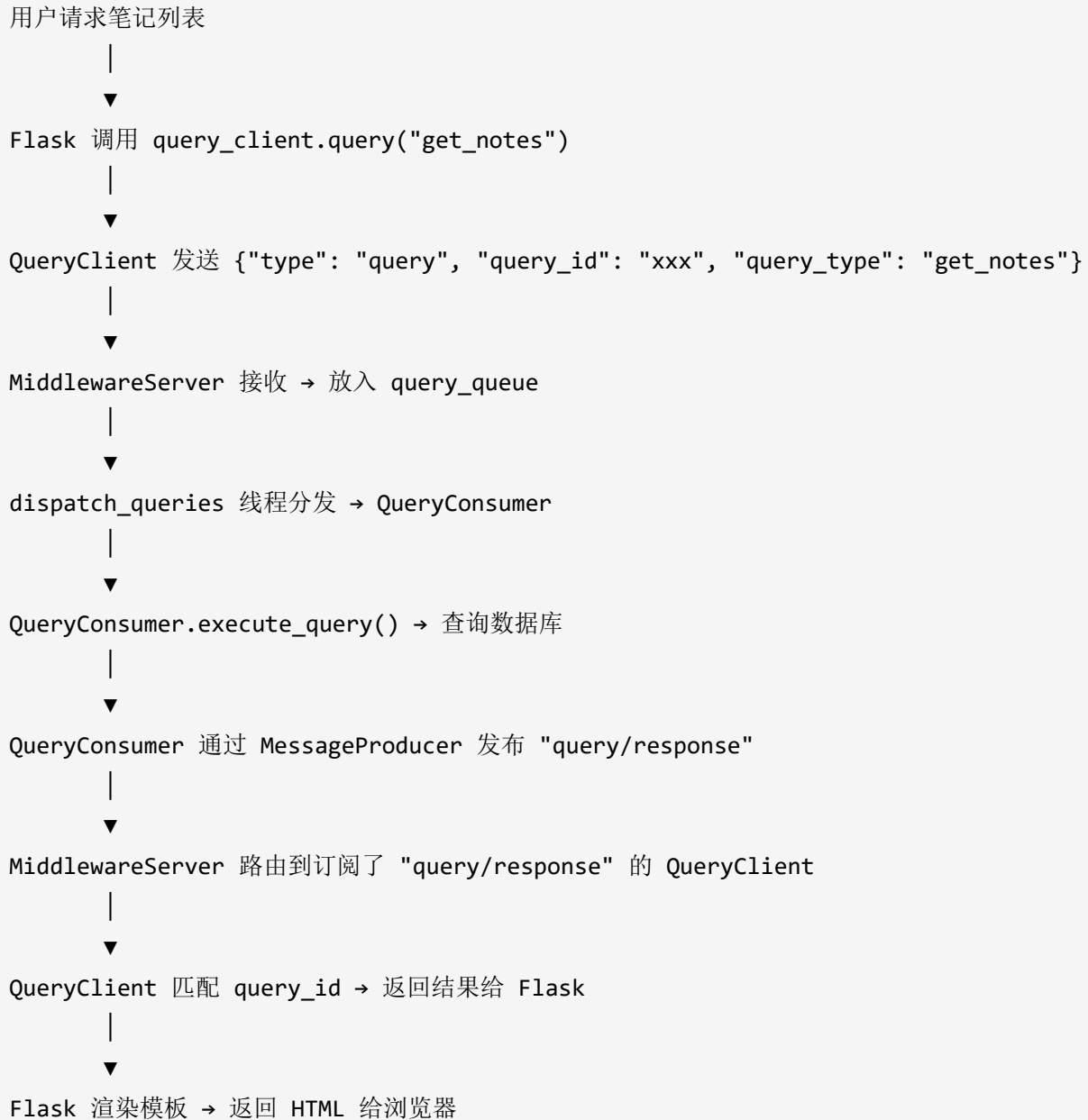


4.5 删除笔记（表单提交）



五、查询流程（异步请求-响应模式）

5.1 流程图



5.2 查询响应匹配机制

```
# QueryClient 使用 query_id 匹配响应
pending_queries = {
    "1779782489.912358": None, # 等待响应
    "1779782490.123456": {"status": "success", "data": [...]} # 已收到
}
```

超时处理: 查询请求超时时间为 10 秒，超时后返回错误响应。

六、项目结构

```
note-sharing-platform/
├── middleware/
│   ├── middleware_server.py
│   ├── message.py
│   ├── message_queue.py
│   └── topic_router.py
├── producer/
│   ├── producer.py
│   └── query_client.py
├── consumers/
│   ├── base_consumer.py
│   ├── audit_consumer.py
│   ├── notice_consumer.py
│   ├── stat_consumer.py
│   └── query_consumer.py
├── web/
│   ├── app.py
│   ├── models.py
│   ├── templates/
│   │   ├── base.html
│   │   ├── index.html
│   │   ├── note_detail.html
│   │   └── add_note.html
│   ├── static/
│   │   └── css/
│   │       └── styles.css
├── utils/
│   └── logger.py
├── data/
├── log/
├── run.py
└── requirements.txt
```

消息中间件核心

主服务器（TCP监听、消息路由）

消息数据结构（序列化/反序列化）

线程安全队列（双队列设计）

主题订阅管理（发布-订阅模式）

消息生产者

事件消息生产者（发布事件）

查询客户端（请求-响应模式）

后台消费者（唯一访问数据库）

消费者基类（连接、订阅、监听）

审核处理器（笔记发布）

通知处理器（点赞、评论）

统计处理器（热度更新）

查询处理器（数据库查询、删除）

Flask Web 应用

应用工厂（路由定义）

数据库模型（SQLite 操作）

HTML 模板

基础模板

首页（笔记列表）

笔记详情页

添加笔记页

静态资源

样式文件

工具模块

日志系统（按分钟目录、组件专属文件）

SQLite 数据库文件

日志输出目录

入口文件

依赖列表

七、架构原则

7.1 完全解耦

- **Flask 与数据库解耦**：通过中间件通信
- **生产者与消费者解耦**：通过主题路由通信
- **中间件与业务解耦**：不处理业务逻辑，只负责路由

7.2 异步处理

- **事件驱动**：所有操作都是异步非阻塞
- **队列缓冲**：消息先入队再处理，削峰填谷
- **并行消费**：多个消费者可同时处理同一主题

7.3 单一职责

- **中间件**：只做消息路由
- **消费者**：只做数据库操作
- **Flask**：只做 HTTP 处理和视图渲染

7.4 可扩展性

- **新增事件类型**：只需添加新主题和消费者
- **水平扩展**：可部署多个中间件节点
- **消费者扩容**：同一主题可添加多个消费者实例

7.5 可靠性

- **线程安全**：队列和路由使用锁保护
 - **持久连接**：生产者和消费者保持长连接
 - **优雅关闭**：支持连接清理和资源释放
-

八、运行说明

8.1 安装依赖

```
pip install -r requirements.txt
```

8.2 启动应用

```
python run.py
```

8.3 服务端口

服务	端口	说明
Flask Web	5000	HTTP 服务
Middleware	5001	消息中间件

九、技术栈

层级	技术	版本	用途
Web 框架	Flask	2.0+	HTTP 处理、模板渲染
数据库	SQLite	内置	轻量级持久化存储
网络通信	Socket	内置	TCP 连接管理
并发处理	threading	内置	多线程支持
日志系统	自定义	-	组件专属日志

许可证

本项目采用 MIT 许可证。